

OVERLOADING OPERATORS

Dr. Subhi A. Rahim
Bahudaila

Lecture 3

OOP

INTRODUCTION

2

Lecture 3

Operator overloading is one of the most exciting features of object-oriented programming. It can transform complex, obscure program listings into obvious ones.

For example, statements like: **d3.addobjects(d1, d2);**
or the similar but equally obscure: **d3 = d1.addobjects(d2);**
can be changed to the much more readable: **d3 = d1 + d2;**

Most of the C++ operators can be overloaded to apply to your new class type.

They are summarized in the following table: →

→ C++ operators

3 Lecture 3

OP	Description
=	the assignment operator
*this	the this pointer
+, -, *, /	arithmetic operators
*= ,+=, ...etc	the arithmetic assignment operators
<, >, <=, >=, ==, and !=	the relational operators
>>, <<	the stream insertion operators
Double, ...etc	a conversion operator,
++, --	the increment operator ++ and the decrement operator --
[]	the subscript operator.

OVERLOADING UNARY OPERATORS (OUO)

4 Lecture 3

- Unary operators act on only one operand.
- Examples of unary operators are the **increment and decrement operators** ++ and --, and the unary minus, as in -33. In the COUNTER example Objects of that class were incremented by calling a member function:

```
c1.inc_count();
```

If we could have used the increment operator ++ instead:

```
++c1;
```

```
// countpp1.cpp
// increment counter variable with ++ operator
```

```
#include <iostream>
using namespace std;
```

```
class Counter
{
```

```
private:
```

```
    unsigned int count; //count
```

```
public:
```

```
    Counter() : count(0) //constructor
```

```
    { }
```

```
    unsigned int get_count() //return count
```

```
    { return count; }
```

```
    void operator ++ () //increment (prefix)
```

```
    { ++count; }
```

```
};
```

```
int main()
```

```
{
```

```
    Counter c1, c2; //define and initialize
```

```
    cout << "\nc1=" << c1.get_count(); //display
```

```
    cout << "\nc2=" << c2.get_count();
```

```
    ++c1; //increment c1
```

```
    ++c2; //increment c2
```

```
    ++c2; //increment c2
```

```
    cout << "\nc1=" << c1.get_count(); //display again
```

```
    cout << "\nc2=" << c2.get_count() << endl;
```

```
    return 0;
```

```
}
```

The keyword **operator** is used to overload the ++ operator in this declarator:
void operator ++ ()

```
c1=0
c2=0
c1=1
c2=2
Press any key to continue.
```

In main() the ++ operator is applied to a specific object, as in the expression **++c1**.

OUO : NAMELESS TEMPORARY OBJECTS

6 Lecture 3

There are more convenient ways to return temporary objects from functions and overloaded operators.

```
Counter operator ++ () //increment count
{
    ++count; // increment count, then return
    return Counter(count); // an unnamed temporary object
} // initialized to this count
```

This statement creates an object of type Counter. This object has no name; it won't be around long enough to need one. This unnamed object is initialized to the value provided by the argument count.

```

// countpp3.cpp
// increment counter variable with ++ operator
// uses unnamed temporary object
#include <iostream>
using namespace std;
////////////////////////////////////
class Counter
{
private:
    unsigned int count; //count
public:
    Counter() : count(0) //constructor no args
    { }
    Counter(int c) : count(c) //constructor, one arg
    { }
    unsigned int get_count() //return count
    { return count; }
    Counter operator ++ () //increment count
    {
        ++count; // increment count, then return
        return Counter(count); // an unnamed temporary object
    } // initialized to this count
};
////////////////////////////////////

int main()
{
    Counter c1, c2; //c1=0, c2=0
    cout << "\nc1=" << c1.get_count(); //display
    cout << "\nc2=" << c2.get_count();
    ++c1; //c1=1
    c2 = ++c1; //c1=2, c2=2
    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count() << endl;
    return 0;
}

```

```

c1=0
c2=0
c1=2
c2=2

```

OUO : POSTFIX NOTATION

8 Lecture 3

So far we've shown the increment operator used only in its prefix form `++c1`. **The postfix is `c1++`**

```

Counter operator ++ (int) //increment count (postfix)
{ //return an unnamed temporary
    return Counter(count++); //object initialized to this
} //count, then increment count

```

There are two different declarators for overloading the ++ operator. The one for prefix notation, is `Counter operator ++ ()` The new one, for postfix notation, is `Counter operator ++ (int)` The only difference is **the int** in the parentheses. This int isn't really an argument, and it doesn't mean integer. **It's simply a signal to the compiler to create the postfix version of the operator.**

```

// postfix.cpp
// overloaded ++ operator in both prefix and postfix
#include <iostream>
using namespace std;
////////////////////////////////////
class Counter
{
private:
    unsigned int count; //count
public:
    Counter() : count(0) //constructor no args
    { }
    Counter(int c) : count(c) //constructor, one arg
    { }
    unsigned int get_count() const //return count
    { return count; }
    Counter operator ++ () //increment count (prefix)
    { //increment count, then return
        return Counter(++count); //an unnamed temporary object
    } //initialized to this count
    Counter operator ++ (int) //increment count (postfix)
    { //return an unnamed temporary
        return Counter(count++); //object initialized to this
    } //count, then increment count
};
////////////////////////////////////
int main()
{
    Counter c1, c2; //c1=0, c2=0
    cout << "\nc1=" << c1.get_count(); //display
    cout << "\nc2=" << c2.get_count();
    ++c1; //c1=1
    c2 = ++c1; //c1=2, c2=2 (prefix)
    cout << "\nc1=" << c1.get_count(); //display
    cout << "\nc2=" << c2.get_count();
    c2 = c1++; //c1=3, c2=2 (postfix)
    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count() << endl;
    return 0;
}

```

```

c1=0
c2=0
c1=2
c2=2
c1=3
c2=2

```

OVERLOADING BINARY OPERATORS

10 Lecture 3

Arithmetic Operators (Binary operators):

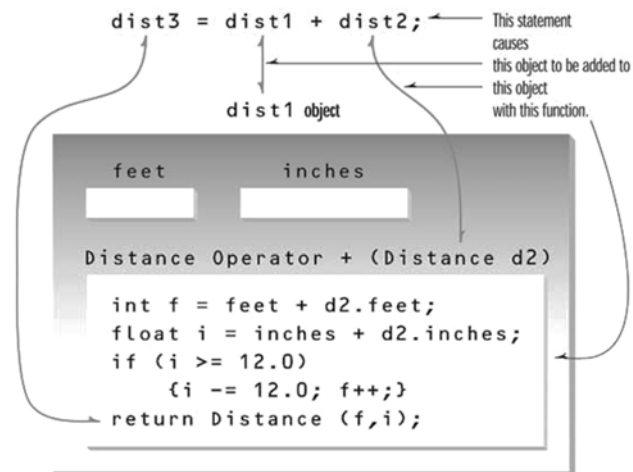


Fig. Overloaded binary operator: one argument

→ Overloading Binary Operators

11 Lecture 3

```
Distance Distance::operator + (Distance d2) const //return sum
{
    int f = feet + d2.feet; //add the feet
    float i = inches + d2.inches; //add the inches
    if(i >= 12.0) //if total exceeds 12.0,
    { //then decrease inches
        i -= 12.0; //by 12.0 and
        f++; //increase feet by 1
    } //return a temporary Distance
    return Distance(f,i); //initialized to sum
}
```

The argument on the left side of the operator (**dist1** in this case) is the object of which the operator is a member. The object on the right side of the operator (**dist2**) must be furnished as an argument to the operator. The operator returns a value, which can be assigned or used in other ways; in this case it is assigned to **dist3**.

12

```
// englplus.cpp
// overloaded '+' operator adds two Distances
#include <iostream>
using namespace std;
////////////////////////////////////
class Distance //English Distance class
{
private:
    int feet;
    float inches;
public: //constructor (no args)
    Distance() : feet(0), inches(0.0)
    {} //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    {}
    void getdist() //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist() const //display distance
    { cout << feet << "\'-" << inches << '\n'; }
    Distance operator + ( Distance ) const; //add 2 distances
};
//add this distance to d2
Distance Distance::operator + (Distance d2) const //return sum
{
    int f = feet + d2.feet; //add the feet
    float i = inches + d2.inches; //add the inches
    if(i >= 12.0) //if total exceeds 12.0,
    { //then decrease inches
        i -= 12.0; //by 12.0 and
        f++; //increase feet by 1
    } //return a temporary Distance
    return Distance(f,i); //initialized to sum
}
```

13 Lecture 3

```

int main()
{
    Distance dist1, dist3, dist4; //define distances
    dist1.getdist(); //get dist1 from user
    Distance dist2(11, 6.25); //define, initialize dist2
    dist3 = dist1 + dist2; //single '+' operator
    dist4 = dist1 + dist2 + dist3; //multiple '+' operators
    //display all lengths
    cout << "dist1 = "; dist1.showdist(); cout << endl;
    cout << "dist2 = "; dist2.showdist(); cout << endl;
    cout << "dist3 = "; dist3.showdist(); cout << endl;
    cout << "dist4 = "; dist4.showdist(); cout << endl;
    return 0;
}

```

The output looks like this:

```

Enter feet: 10
Enter inches: 6.5
dist1 = 10'-6.5"
dist2 = 11'-6.25"
dist3 = 22'-0.75"
dist4 = 44'-1.5"
Press any key to continue

```

COMPARISON OPERATORS

14 Lecture 3

□ Comparing Distances:

Example we'll **overload the less than operator (<)** in the Distance class so that we can compare two distances.

```

bool Distance::operator < (Distance d2) const //return the sum
{
    float bf1 = feet + inches/12;
    float bf2 = d2.feet + d2.inches/12;
    return (bf1 < bf2) ? true : false;
}

```

The **operator<()** function has a return type of **bool**. The return value is **false** or **true**, depending on the comparison of the two distances.

return (bf1 < bf2) ? true : false;

is the same as

if(bf1 < bf2) return true;

else return false;

```

// engless.cpp
// overloaded '<' operator compares two Distances
#include <iostream>
using namespace std;
15 //////////////////////////////////////////////////
class Distance //English Distance class
{
private:
    int feet;
    float inches;
public: //constructor (no args)
    Distance() : feet(0), inches(0.0)
    { } //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
    void getdist() //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist() const //display distance
    { cout << feet << "'-" << inches << "'\n"; }
    bool operator < (Distance) const; //compare distances
};
////////////////////////////////////////////////
//compare this distance with d2
bool Distance::operator < (Distance d2) const //return the sum
{
    float bf1 = feet + inches/12;
    float bf2 = d2.feet + d2.inches/12;
    return (bf1 < bf2) ? true : false;
}
////////////////////////////////////////////////

```

16 Lecture 3

```

int main()
{
    Distance dist1; //define Distance dist1
    dist1.getdist(); //get dist1 from user
    Distance dist2(6, 2.5); //define and initialize dist2
    //display distances
    cout << "\ndist1 = "; dist1.showdist();
    cout << "\ndist2 = "; dist2.showdist();
    if( dist1 < dist2 ) //overloaded '<' operator
        cout << "\ndist1 is less than dist2";
    else
        cout << "\ndist1 is greater than (or equal to) dist2";
    cout << endl;
    return 0;
}

```

```

Enter feet: 5
Enter inches: 11.5

dist1 = 5'-11.5"
dist2 = 6'-2.5"
dist1 is less than dist2
Press any key to continue

```

→ Comparing Strings

17

Lecture 3

- Example of overloading an operator, the equal to (==) operator.

```
bool operator == (String ss) const //check for equality
{ return ( strcmp(str, ss.str)==0 ) ? true : false;}
```

The **operator==()** function uses the library function **strcmp()** to compare the two C-strings.

This function returns 0 if the strings are equal, a negative number if the first is less than the second, and a positive number if the first is greater than the second.

18

```
//strequal.cpp
//overloaded '==' operator compares strings
#include <iostream>
using namespace std;
#include <string.h> //for strcmp()
////////////////////////////////////
class String //user-defined string type
{
private:
    enum { SZ = 80 }; //size of String objects
    char str[SZ]; //holds a string
public:
    String() //constructor, no args
    { strcpy(str, ""); }
    String( char s[] ) //constructor, one arg
    { strcpy(str, s); }
    void display() const //display a String
    { cout << str; }
    void getstr() //read a string
    { cin.get(str, SZ); }
    bool operator == (String ss) const //check for equality
    { return ( strcmp(str, ss.str)==0 ) ? true : false; }
};
////////////////////////////////////
int main()
{
    String s1 = "yes";
    String s2 = "no";
    String s3;
    cout << "\nEnter 'yes' or 'no': ";
    s3.getstr(); //get String from user
    if(s3==s1) //compare with yes
        cout << "You typed yes\n";
    else if(s3==s2) //compare with no
        cout << "You typed no\n";
    else
        cout << "You didn't follow instructions\n";
    return 0;
}
```

The output :

```
Enter 'yes' or 'no': yes
You typed yes
Press any key to continue_
```

ARITHMETIC ASSIGNMENT OPERATORS

19

Lecture 3

- The arithmetic assignment operator: the += operator. Recall that this operator combines assignment and addition into one step

```
void Distance::operator += (Distance d2)
{
    feet += d2.feet; //add the feet
    inches += d2.inches; //add the inches
    if(inches >= 12.0) //if total exceeds 12.0,
    { //then decrease inches
        inches -= 12.0; //by 12.0 and
        feet++; //increase feet
    } //by 1
}
```

In the **operator+=()** function in ENGLPLEQ, the object that takes on the value of the sum is the object of which the function is a member. The **operator+=()** function has no return value; it returns type void. A return value is not necessary with arithmetic assignment operators such as +=, because the result of the assignment operator is not assigned to anything.

20

```
// englpleq.cpp
// overloaded '+' assignment operator
#include <iostream>
using namespace std;
////////////////////////////////////
class Distance //English Distance class
{
private:
    int feet;
    float inches;
public: //constructor (no args)
    Distance() : feet(0), inches(0.0)
    { } //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
    void getdist() //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist() const //display distance
    { cout << feet << "\'-" << inches << '\n'; }
    void operator += ( Distance );
};
//-----
//add distance to this one
void Distance::operator += (Distance d2)
{
    feet += d2.feet; //add the feet
    inches += d2.inches; //add the inches
    if(inches >= 12.0) //if total exceeds 12.0,
    { //then decrease inches
        inches -= 12.0; //by 12.0 and
        feet++; //increase feet
    } //by 1
}
////////////////////////////////////
```

21 Lecture 3

```

int main()
{
    Distance dist1; //define dist1
    dist1.getdist(); //get dist1 from user
    cout << "\ndist1 = "; dist1.showdist();
    Distance dist2(11, 6.25); //define, initialize dist2
    cout << "\ndist2 = "; dist2.showdist();
    dist1 += dist2; //dist1 = dist1 + dist2
    cout << "\nAfter addition, ";
    cout << "\ndist1 = "; dist1.showdist();
    cout << endl;
    return 0;
}

```

```

Enter feet: 3
Enter inches: 5.75

dist1 = 3'-5.75"
dist2 = 11'-6.25"
After addition,
dist1 = 15'-0"
Press any key to continue

```

THE SUBSCRIPT OPERATOR ([])

22 Lecture 3

The symbol `[ ]` denotes the subscript operator. Its name derives from the original use of arrays, where a `[ i ]` represented the mathematical symbol a_i . When used as a `[ i ]`, it has two operands: `a` and `i`. The expression `a [ i ]` is equivalent to operator `[ ] (a, i)`. And as an operator, `[ ]` can be overloaded.

```

int main()
{
    safearray sal;
    for(int j=0; j<LIMIT; j++) //insert elements
        sal[j] = j*10; //left side of equal sign
    for(j=0; j<LIMIT; j++) //display elements
    {
        int temp = sal[j]; //right side of equal sign
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}

feet++; //increase feet
} //by 1
}
return 0;
}

```

In this program we can use the natural subscript expressions
`sal[j] = j*10;` and
`temp = sal[j];`
 for input and output to the safe array.

```

// arrow3.cpp
// creates safe array (index values are checked before access)
// uses overloaded [] operator for both put and get
#include <iostream>
using namespace std;
23 #include <process.h> //for exit()
const int LIMIT = 10; //array size
////////////////////////////////////
class safearray
{
private:
    int arr[LIMIT];
public:
    int& operator [](int n) //note: return by reference
    {
        if( n< 0 || n>=LIMIT )
        { cout << "\nIndex out of bounds"; exit(1); }
        return arr[n];
    }
};
////////////////////////////////////
int main()
{
    safearray sa1;
    for(int j=0; j<LIMIT; j++) //insert element
        sa1[j] = j*10; //left side of equation
    for(j=0; j<LIMIT; j++) //display element
    {
        int temp = sa1[j]; //right side of equation
        cout << "Element " << j << " is " << temp << endl;
    }
    return 0;
}

```

```

Element 0 is 0
Element 1 is 10
Element 2 is 20
Element 3 is 30
Element 4 is 40
Element 5 is 50
Element 6 is 60
Element 7 is 70
Element 8 is 80
Element 9 is 90
Press any key to continue.

```